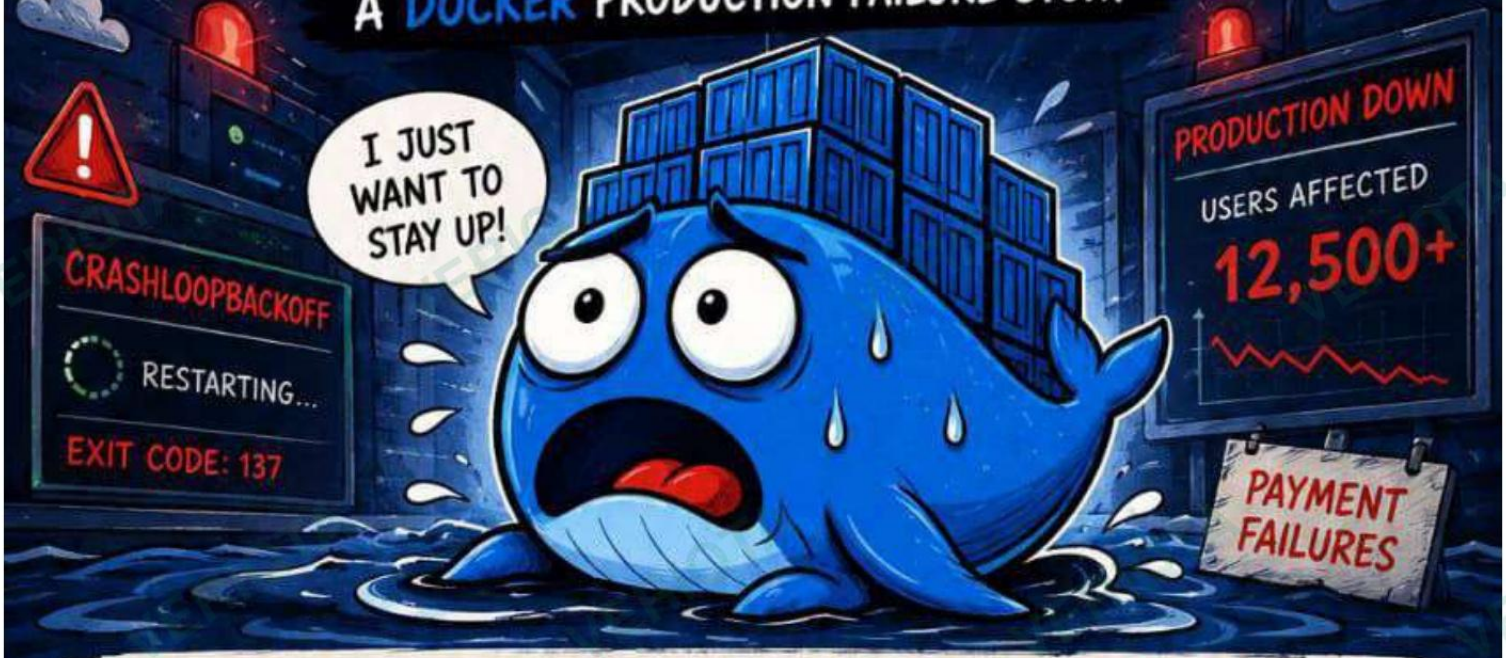
SERIES 6
PRODUCTION CASE FILE

VERIQTA
BUILDING ENGINEERS. BUILDING FUTURES.

CASE FILE #006

THE CONTAINER THAT KEPT CRASHING!!

A DOCKER PRODUCTION FAILURE STORY



10 REAL MISTAKES. 1 MAJOR OUTAGE. ZERO EXCUSES.

PRODUCTION LESSONS | REAL EVIDENCE & INVESTIGATION | PRACTICAL FIXES & EXAMPLES | RELIABILITY THAT SCALES | INTERVIEW READY KNOWLEDGE

WHAT'S INSIDE

- From CrashLoopBackOff to Root Cause
- Docker & Kubernetes Mistakes We Made
- Step-by-Step Investigation with Real Commands
- Best Practices & Production-Ready Fixes
- Architecture, Monitoring & Observability
- Actionable Lessons for Every Engineer

YOUR GUIDE TO BUILDING RELIABLE, RESILIENT SYSTEMS IN PRODUCTION.

INCIDENT DATE
MAY 28, 2026

DURATION
~2h 47m

SEVERITY
SEV-1 (CRITICAL)

AFFECTED SERVICE
PAYMENT GATEWAY

REAL OUTAGE. REAL LESSONS. REAL IMPACT.

BECAUSE IN PRODUCTION, SMALL MISTAKES CAN BRING YOUR ENTIRE SYSTEM TO ITS KNEES.

VERIQTA
ING ENGINEERS. BUILDING FUTUR

SERIES 6

VERIQTA

BUILDING ENGINEERS. BUILDING FUTURES.

PRODUCTION CASE FILE #006

THE CONTAINER THAT KEPT CRASHING !!

A Docker Production Failure Story

What started as a simple deployment turned into a critical production incident when our containers refused to stay alive. Let's investigate what really happened and what we learned.

IMPACT

- 47% Payment failures across all regions
- 12,500+ Users affected
- 3 Regions impacted
- Revenue loss detected

INCIDENT OVERVIEW

- DATE: May 28, 2026
- START TIME: 02:14 AM (UTC)
- DURATION: ~2h 47m
- SEVERITY: SEV-1 (Critical)
- AFFECTED SERVICE: Payment Gateway

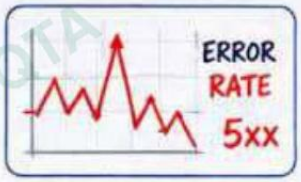
WHAT HAPPENED?

- 02:14 AM **Alerts triggered** — High error rate detected in payment service
- 02:16 AM **Pods restarting** — Payment pods in CrashLoopBackOff
- 02:25 AM **Traffic drops** — Automatic retries increase user impact
- 02:40 AM **Engineers engaged** — Incident declared SEV-1
- 04:10 AM **Root cause identified** — Multiple Docker misconfigurations
- 04:61 AM **Service restored** — Stabilized and traffic normalized

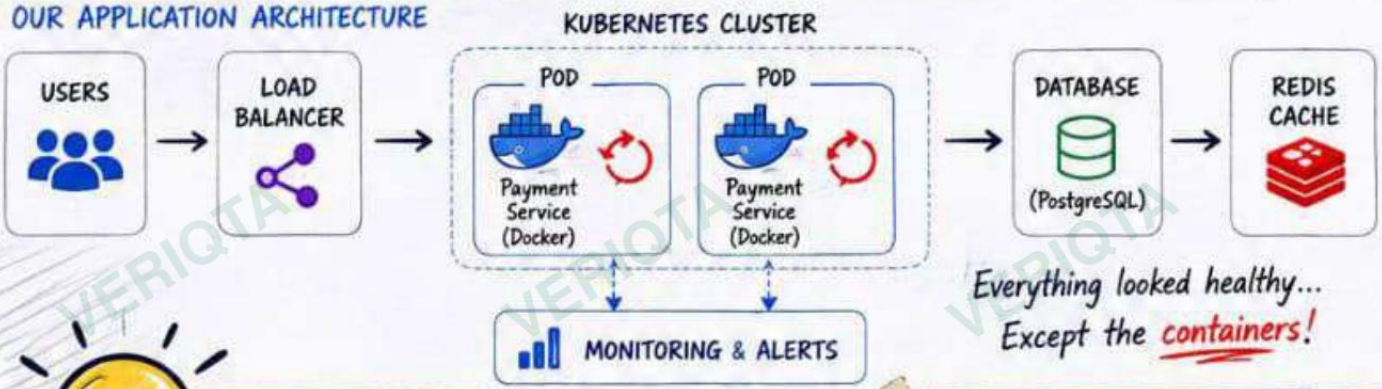
INITIAL SYMPTOMS

- High HTTP 5xx error rate
- Payment API timing out
- Kubernetes pods restarting
- No obvious CPU spike
- Memory "normal" (or so it seemed)
- Infrastructure healthy
- Database healthy

→ But containers kept dying!



OUR APPLICATION ARCHITECTURE



Everything looked healthy... Except the containers!

LESSON SO FAR:

In production, the smallest misconfiguration in your Docker images can bring your entire system to its knees.

Our mission in this case file: Find the rea… and prevent…

VERIQTA
…G ENGINEERS. BUILDING FUTUR

MISTAKE #1

RUNNING AS ROOT

The dangerous default that caused more than just a security risk.

Our Docker container was running as the root user (UID 0). It worked in staging. It failed in production. Running as root opened the door to security risks and also contributed to instability and unexpected behavior in our environment.

DANGER ZONE

Root inside a container means root on the host can be a single exploit away.

Don't give attackers the keys to the kingdom!

WHY THIS IS A PROBLEM

Security Risk
An attacker who breaks out of the container gets root privileges on the host.

File System Damage
Root can modify or delete critical system files, causing instability.

Privilege Escalation
Easier to escalate privileges inside the container and on the underlying node.

Breaks Least Privilege
Containers should be isolated and run with the minimum privileges possible.

Contributes to Instability
Root-owned files, processes, and sockets can cause permission conflicts and crashes.

EVIDENCE FROM OUR ENVIRONMENT

```
$ kubectl exec -it payment-service-7d6f8c9f7d-abc12 -- whoami
root
$ id
uid=0(root) gid=0(root) groups=0(root)
$ ls -l /app
total 32
-rw-r--r--  1 root root  4096 May 28 01:12 server.js
-rw-r--r--  1 root root  1024 May 28 01:12 config.json
drwxr-xr-x  2 root root  4096 May 28 01:12 logs
$ ps aux | head -3
USER      PID %CPU %MEM    VSZ   RSS TTY   STAT START   TIME COMMAND
root        1  0.1  1.2 16700  6540 ?     Ss   01:12   0:00 node server.js
...
```

⚠ Container is running as root (UID 0)

HOW THIS CAUSED OUR INCIDENT

1. A vulnerability in a dependency was exploited.
2. Attacker gained shell access inside the container.
3. Since the container ran as root, attacker had full control.
4. Attacker modified critical files and filled /tmp with junk data.
5. Processes started failing due to permission errors and resource exhaustion.
6. Main process crashed. Kubernetes restarted the pod.
7. CrashLoopBackOff began.

REAL IMPACT ON OUR SYSTEM

Security Exposure
Full host compromise was possible. We were one exploit away from a total breach.

System Instability
Permission conflicts and unexpected file changes made the container unstable.

Downtime & Restarts
Pods kept restarting (CrashLoopBackOff), causing intermittent payment failures.

Business Impact
Failed payments, unhappy customers, and loss of revenue.

WHAT WE FOUND

- Dockerfile did not specify a non-root user.
- Application files were owned by root.
- Write operations in /tmp and /var were done by root.
- Some libraries behaved differently when running as root.
- Kubernetes security context was not defined.

THE RIGHT WAY: RUN AS NON-ROOT USER

```
# Create a non-root user
RUN groupadd -r appuser && useradd -r -g appuser appuser

# Set working directory
WORKDIR /app

# Copy files and change ownership
COPY --chown=appuser:appuser . /app

# Switch to non-root user
USER appuser

# (Optional) Add security context in Kubernetes
securityContext:
  runAsUser: 10001
  runAsNonRoot: true
  readOnlyRootFilesystem: true
```

BEST PRACTICES

- Always run containers as non-root.
- Use a dedicated user with least privileges.
- Set proper file permissions and ownership.
- Use read-only root filesystem when possible.
- Drop all unnecessary Linux capabilities.
- Define securityContext in Kubernetes.
- Continuously scan images for privilege issues.

KEY TAKEAWAY

Running as root in containers is a ticking time bomb.
It's not just about security—it
isolation, and reliability.

Make non-root your defa

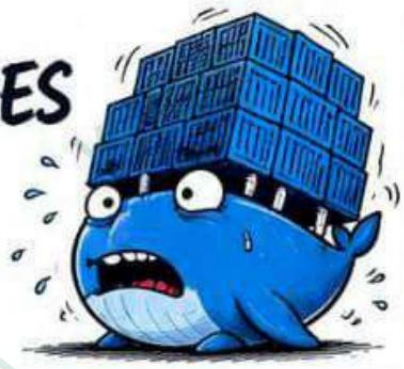
VERIQTA
NG ENGINEERS. BUILDING FUTUR

MISTAKE #2

HUGE CONTAINER IMAGES

Slow, Heavy, and Hard to Recover

Our payment-service image was **2.41GB.**
It contained unnecessary tools, build dependencies, and caches that had no place in production.
This single mistake slowed deployments, increased recovery time, and made every restart painful.



DANGER ZONE

Bigger images mean:

- Slower deployments
- Longer rollbacks
- Slower scaling
- Higher registry cost
- Larger attack surface

Every MB matters in production!

WHY THIS IS A PROBLEM

Slow Deployments
Large images take longer to pull, especially across regions.

Slow Recovery
When pods crash, Kubernates has to pull the image again. Big images = more downtime.

Slower Scaling
Auto-scaling becomes slow and unreliable with huge images.

Higher Costs
More bandwidth, more registry storage, more time, more money.

Larger Attack Surface
More packages = more vulnerabilities.
More bloat = more risk.

EVIDENCE FROM OUR ENVIRONMENT

```
$ docker images payment-service
REPOSITORY        TAG       IMAGE ID       SIZE      CREATED
payment-service   latest    a1b2c3d4e5f6   2.41GB    5 days ago

$ docker history payment-service --no-trunc | head -n 10
IMAGE          CREATED      CREATED BY                          SIZE     COMMENT
a1b2c3d4e5f6   5 days ago   /bin/sh -c npm run build            1.21GB   build artifacts
b2c3d4e5f6a7   5 days ago   /bin/sh -c apt-get install ...      512MB    build deps
c3d4e5f6a7b8   5 days ago   /bin/sh -c curl https://...         312MB    tools
...
```

⚠ Image is 2.41GB — 8x bigger than it needs to be.

REAL IMPACT ON OUR INCIDENT

Longer Pull Time
Took ~96 seconds to pull the image in us-east-1.
In other regions: 2-4 minutes!

Slower Recovery
Every crash triggered a full image pull.
Recovery time increased by ~300%.

Deployment Pain
Rolling update of 3 pods took ~8 minutes.
Rollback took even longer.

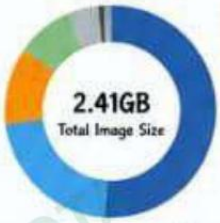
Business Impact
During restarts and slow rollouts, customers saw failures.
Revenue impacted.

HOW WE DISCOVERED IT

1. 10:15 AM — Checked pod events. Image pull time was unusually high.
2. 10:22 AM — Timed image pull manually. Took 96 seconds in the same region!
3. 10:28 AM — Inspected image size with docker images. Found the image was 2.41GB.
4. 10:35 AM — Inspected image layers with docker history. Found build tools, caches, and docs.
5. 10:42 AM — Reviewed Dockerfile. Using a single-stage build.
6. 10:55 AM — Root cause confirmed: Image bloat was making everything slower.

WHAT WE FOUND INSIDE THE IMAGE

- Node.js build dependencies (devDependencies)
- Package managers (apt, curl, git, vim)
- Build tools (gcc, make, python)
- Cache directories (npm cache, apt cache)
- Documentation and examples
- Tests and test data
- Unnecessary locales and man pages



Build Artifacts	1.21GB	(50%)
Node Modules	512MB	(21%)
Tools & Utilities	312MB	(13%)
Caches	256MB	(11%)
Docs & Tests	96MB	(4%)
Other	44MB	(2%)

80% of the image was unnecessary!

THE RIGHT WAY: USE MULTI-STAGE BUILDS

BEFORE (Single Stage)

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install      # installs all deps (including dev)
RUN npm run build    # build app
CMD ["node", "server.js"]
```

IMAGE SIZE: 2.41GB

AFTER (Multi-Stage)

```
# Build Stage
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Production Stage
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package*.json ./
CMD ["node", "dist/server.js"]
```

IMAGE SIZE: 312MB (87% SMALLER!)

BEST PRACTICES

- Always use multi-stage builds.
- Copy only what you need to the final image.
- Use .dockerignore to exclude unnecessary files.
- Use slim base images (alpine/distroless).
- Remove package caches and temp files.
- Avoid installing debugging tools in prod images.
- Scan images for vulnerabilities.

KEY TAKEAWAY

A smaller image equals faster deployments, faster recovery, lower costs, and a smaller attack surface. Build lean, ship fast, sleep well.

Small images. Big impact.

LESSON SO FAR:

Running as root opened the door.
Huge images slowed down everything.
Next, we'll see how missing health checks made the problem even worse.

NEXT UP:

Mistake #3:
Missing Health Checks

SERIES PROGRESS

1 Run as Root — 2 Huge Images

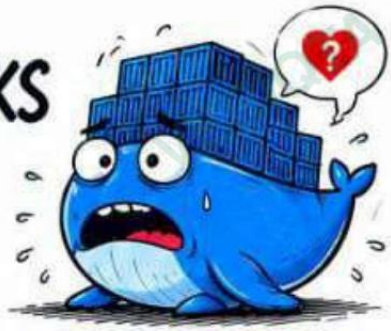
VERIQTA
ENGINEERS. BUILDING FUTURE

MISTAKE #3

MISSING HEALTH CHECKS

Kubernetes had no idea we were failing

Our container didn't have proper health checks.
Kubernetes thought everything was fine,
even when the application was already failing.
Users saw errors first. Monitoring saw failures later.
This made the outage longer and harder to detect.



DANGER ZONE

No health checks mean:

- Kubernetes can't detect failures
- Traffic goes to broken pods
- Users see errors before we do
- Restarts take longer to trigger
- More failed requests & retries
- Longer mean time to recover

Blind containers are expensive!

WHY THIS IS A PROBLEM

Kubernetes is Blind
Without health checks, Kubernetes thinks the container is healthy.

Traffic to Broken Pods
Load balancer keeps sending traffic to pods that are already unhealthy.

Late Detection
Failures are detected much later, increasing downtime.

Slow Recovery
Kubernetes restarts unhealthy pods only after it knows they're dead.

Business Impact
More failed requests, unhappy customers, and lost revenue.

EVIDENCE FROM OUR ENVIRONMENT

```
$ kubectl describe pod payment-service-7d6f8c9f7d-abc12 | grep -A 2 -i readiness
Warning  Unhealthy  45m (x312 over 2h)  kubelet  Readiness probe failed: HTTP probe
failed with statuscode: 500

$ kubectl describe pod payment-service-7d6f8c9f7d-abc12 | grep -A 2 -i liveness
Warning  Unhealthy  45m (x189 over 2h)  kubelet  Liveness probe failed: HTTP probe
failed with statuscode: 500

$ kubectl get pods -w
NAME                                  READY   STATUS             RESTARTS   AGE
payment-service-7d6f8c9f7d-abc12      1/1     Running            23         45m
payment-service-7d6f8c9f7d-def34      1/1     Running            21         45m
payment-service-7d6f8c9f7d-ghi56      0/1     CrashLoopBackOff   47         45m
payment-service-7d6f8c9f7d-jkl78      1/1     Running            20         45m
...
```

⚠ Probes were not defined in the pod spec.

HOW WE DISCOVERED IT

1. 10:55 AM — Noticed high 5xx errors from users.
2. 11:00 AM — Checked monitoring dashboards. App was failing but pods were "healthy".
3. 11:05 AM — Described a failing pod. No readiness/liveness probes configured.
4. 11:10 AM — Checked with team. Health checks were never added.
5. 11:15 AM — Confirmed by reviewing deployment YAML. No probes defined.
6. 11:20 AM — Root cause confirmed: Kubernetes had no way to detect failures.

WHAT WE SAW IN REAL TIME

- Users were seeing errors for ~20 minutes before any alerts fired!
- Without health checks, we were flying blind. This cost us time and money.

IMPACT ON THIS INCIDENT

~20 minutes
Users saw failures before we knew anything was wrong.

47 restarts
One pod restarted 47 times during the incident.

More failed requests
Broken pods kept receiving traffic, causing more 5xx errors.

Longer outage
Recovery was delayed significantly.

THE FIX: ADD HEALTH CHECKS

BEFORE (No Probes)

```
containers:
- name: payment-service
  image: registry.example.com/
    payment-service:1.0
  ports:
  - containerPort: 8080

  # No readinessProbe
  # No livenessProbe
```

→

AFTER (With Probes)

```
containers:
- name: payment-service
  image: registry.example.com/
    payment-service:1.0
  ports:
  - containerPort: 8080
  readinessProbe:
    httpGet:
      path: /health/ready
      port: 8080
    initialDelaySeconds: 10
    periodSeconds: 10
  livenessProbe:
    httpGet:
      path: /health/live
      port: 8080
    initialDelaySeconds: 10
    periodSeconds: 15
    failureThreshold: 3
```

WHAT WE CHECK NOW

- Readiness probe before sending traffic
- Liveness probe to detect deadlocks/crashes
- Proper timeouts and thresholds
- Endpoint returns fast and reliably
- Separate /ready and /live endpoints
- Monitor probe failures as critical alerts

EXAMPLE HEALTH ENDPOINTS (Node.js)

```
// /health/live
app.get('/health/live', (req, res) => {
  res.status(200).send('OK');
});

// /health/ready
app.get('/health/ready', async (req, res) => {
  const db = await checkDatabase();
  const redis = await checkRedis();
  if (db && redis) res.status(200).send('READY');
  else res.status(503).send('NOT READY');
});
```

KEY TAKEAWAY

Health checks are not optional.
They are your early warning system.
If Kubernetes can't see it, it can't fix it.

BENEFITS AFTER THE FIX

- Kubernetes removes bad pods from service
- Users are not sent to failing pods
- Failures are detected in seconds
- Faster restarts and recovery
- Less downtime, happier customers

REAL LESSON

Don't just run containers.
Make them observable.
Make them self-aware.
Health checks save outages.

SO FAR IN THIS SERIES

- #1 Running as Root
- #2 Huge Container Images
- #3 Missing Health Checks

NEXT UP

Mistake #4:
Wrong Resource Limits
(CPU & Memory)

SERIES PR...

1 Run as Root — 2 Huge Images — 3 No Health Checks — Wrong Limits — More to Come

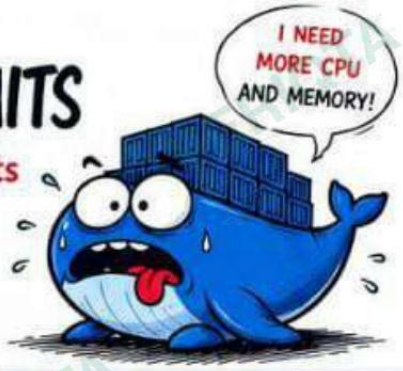
VERIQTA
...G ENGINEERS. BUILDING FUTUR...

MISTAKE #4

WRONG RESOURCE LIMITS

Starving the App and Triggering Restarts

Our containers had no or incorrect resource limits.
During traffic spikes, the application was being OOMKilled and throttled constantly. Kubernetes restarted the pods, but the real problem was resource starvation.
More resources ≠ Better. Right resources = Stability.



DANGER ZONE

Wrong limits lead to:

- OOMKills
- CPU throttling
- High latency
- Pod restarts
- Unpredictable performance
- Angry users

Resources are not unlimited!

WHY THIS IS A PROBLEM

CPU Throttling
CPU limit too low → throttling. App slows down and times out.

OOMKills
Memory limit too low → container killed by kernel. Pod restarts.

No Limits = Noisy Neighbor
One pod can consume everything and crash others.

Slow Recovery
Every restart adds delay and increases user impact.

Business Impact
Higher latency, failed payments, support tickets, and lost trust.

EVIDENCE FROM OUR ENVIRONMENT

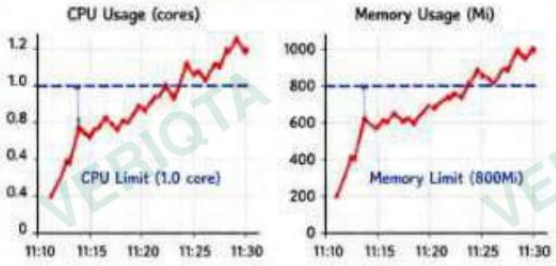
```
$ kubectl top pod -n payments
NAME                                  CPU(cores)      MEMORY(bytes)
payment-service-7d6f8c9f7d-abc12      950m  (95%)     780Mi  (97%)
payment-service-7d6f8c9f7d-ef34d      1020m (102%)    810Mi  (101%)
payment-service-7d6f8c9f7d-ghi56      980m  (98%)     790Mi  (98%)

$ kubectl get events -n payments --sort-by=.lastTimestamp | grep -i kill
Warning  OOMKilled     45m  kubelet   Container payment-service was killed
Warning  BackOff       45m  kubelet   Back-off restarting failed container
Warning  FailedProbe   45m  kubelet   Liveness probe failed: HTTP probe failed
Warning  Unhealthy     45m  kubelet   Readiness probe failed: HTTP probe failed
```

⚠ Pods are out of resources and getting killed!

HOW WE DISCOVERED IT

1. 11:25 AM — Noticed high latency during traffic spike.
2. 11:26 AM — Checked pod restarts. Count was increasing.
3. 11:27 AM — Checked pod events. Saw OOMKilled errors.
4. 11:28 AM — Checked resource usage with metrics-server.
5. 11:30 AM — Found CPU 100%+ and Memory 97%+.
6. 11:32 AM — Confirmed limits were too low in deployment YAML.

METRICS AT THE TIME



- CPU usage was hitting the limit and getting throttled.
- Memory usage crossed the limit → OOMKills.
- Restarts increased every 1–2 minutes.

IMPACT ON THIS INCIDENT

CPU Throttling
Requests were too low. App was throttled badly.

OOMKills
Memory limit was too low. Kernel killed the container.

47 Restarts
Pods restarted 47 times during the incident.

High Latency
P95 latency went from 120ms to 4.2s.

Revenue Impacted
Failed payments and timeouts led to revenue loss.

THE FIX: SET RIGHT REQUESTS & LIMITS

BEFORE (Wrong Limits)

```
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

Too low → throttling & OOMKills

AFTER (Right Limits)

```
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
  limits:
    cpu: "1000m"
    memory: "1024Mi"
```

Right size → stable & predictable

HOW WE DECIDED THE RIGHT VALUES

- Checked historical usage (95th percentile).
- Added 20–30% buffer for spikes.
- Set requests = what the app needs to run.
- Set limits = max it can use when needed.
- Tested with load and validated stability.

TOOLS WE USED

- metrics-server (kubectl top)
- Prometheus + Grafana
- Kubernetes Events
- Load Testing (k6)
- Application Logs

RESOURCE TUNING PRINCIPLES

- Requests should reflect real usage.
- Limits should allow headroom, not be too tight.
- Avoid setting requests = limits.
- Tune based on metrics, not guesses.
- Revisit and tune periodically.

RESULT AFTER THE FIX

- 0 OOMKills after fix
- 0 CPU Throttling after fix
- Restarts after fix
- Latency Stable — P95 ~ 150ms
- System Stable — No more user facing errors

KEY TAKEAWAY

Kubernetes doesn't know your app's needs.
It enforces what you define.
Right resource limits bring stability, performance, and happy users.
Measure. Understand. Configure.

LESSON SO FAR

- #1 Running as Root
- #2 Huge Container Images
- #3 Missing Health Checks
- #4 Wrong Resource Limits

NEXT UP

Mistake #5:
Missing Graceful Shutdown (SIGTERM Handling)

SERIES PROGRESS

1 Run as Root — 2 Huge Images — N… Checks — … Limits — … to Come

VERIQTA ENGINEERS. BUILDING FUTUR

MISTAKE #5

MISSING GRACEFUL SHUTDOWN

Kubernetes Killed It. We Didn't Handle It.

Our application didn't handle shutdown signals.
When Kubernetes needed to restart the pod,
it sent SIGTERM and waited... but our app ignored it.
After the grace period, Kubernetes sent SIGKILL.
Data was lost. Requests failed. Users suffered.

DANGER ZONE

No graceful shutdown leads to:

- Incomplete requests
- Corrupted data
- Lost in-flight transactions
- Broken connections
- Unhappy users

Shutdown is part of reliability!

WHY THIS IS A PROBLEM

SIGTERM Ignored
App keeps running like nothing happened.

Forced SIGKILL
Kubernetes has no choice but to kill the container.

Requests Are Dropped
In-flight requests are terminated mid-way.

Data Can Be Lost
No time to flush buffers, close connections, or save state.

More Restarts, More Pain
Failed shutdowns lead to unstable pods and more restarts.

EVIDENCE FROM OUR ENVIRONMENT

```
$ kubectl describe pod payment-service-7d6f8c9f7d-ghi56 | grep -A 20 "Events:"
```

Type	Reason	Age	From	Message
Normal	Killing	45m	kubelet	Stopping container payment-service
Normal	Killing	45m	kubelet	Container payment-service (PID 1) received SIGTERM
Warning	FailedPreStop	45m	kubelet	PreStop hook failed: context deadline exceeded
Warning	Unhealthy	45m	kubelet	Readiness probe failed: HTTP probe failed
Normal	Killing	45m	kubelet	Container payment-service failed to stop in 30s
Normal	Killing	45m	kubelet	Container payment-service force killed with SIGKILL
Normal	BackOff	45m	kubelet	Back-off restarting failed container payment-service

```
$ kubectl get pods -w
```

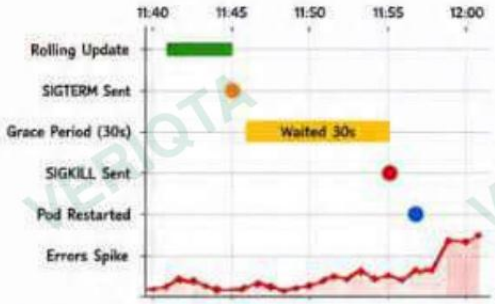
NAME	READY	STATUS	RESTARTS	AGE
payment-service-7d6f8c9f7d-ghi56	1/1	Running	48	47m
payment-service-7d6f8c9f7d-ghi56	0/1	Terminating	48	47m
payment-service-7d6f8c9f7d-jkl78	0/1	Pending	0	0s
payment-service-7d6f8c9f7d-jkl78	0/1	ContainerCreating	0	1s
payment-service-7d6f8c9f7d-jkl78	0/1	CrashLoopBackOff	1	5s

⚠ Our app didn't handle SIGTERM. Kubernetes had to kill it.

HOW WE DISCOVERED IT

1. 11:45 AM — Noticed errors during rolling update.
2. 11:47 AM — Saw pods entering Terminating state.
3. 11:48 AM — Checked pod events. Found SIGTERM → SIGKILL.
4. 11:50 AM — Reviewed application logs. No shutdown logs found.
5. 11:52 AM — Reproduced locally with docker stop. App was killed before cleanup.
6. 11:55 AM — Confirmed app didn't trap shutdown signals. Root cause confirmed.

WHAT WE SAW IN REAL TIME



- App ignored SIGTERM.
- Kubernetes waited 30s (terminationGracePeriodSeconds).
- Then sent SIGKILL.
- Pod restarted. Users saw errors.

IMPACT ON THIS INCIDENT

~12 minutes
Errors increased during rolling update due to forced kills and restarts.

23 restarts
Additional restarts caused by bad shutdown handling.

Risk of Data Loss
In-flight requests and partial transactions were lost.

User Experience
Users saw timeouts, 5xx errors, and payment failures.

Revenue Impact
Failed payments and retries led to loss of revenue.

THE FIX: HANDLE SHUTDOWN GRACEFULLY

1. TRAP SIGNALS IN THE APPLICATION (Node.js Example)

```
process.on('SIGTERM', async () => {
  console.log('SIGTERM received. Shutting down gracefully...');
  server.close(() => {
    console.log('HTTP server closed.');
  });
  await closeDatabase();
  await flushMetrics();
  console.log('Cleanup complete. Exiting.');
  process.exit(0);
});

process.on('SIGINT', async () => {
  console.log('SIGINT received.');
  process.emit('SIGTERM');
});
```

2. ADD A PRESTOP HOOK (Kubernetes)

```
lifecycle:
  preStop:
    exec:
      command: ["/bin/sh", "-c", "sleep 5"]
```

Gives the app a few extra seconds to finish in-flight work.

3. SET A PROPER GRACE PERIOD

```
spec:
  terminationGracePeriodSeconds: 30
```

Give enough time for your app to finish cleanly.

WHAT WE ADDED TO OUR APP

- Signal handlers (SIGTERM, SIGINT)
- HTTP server close
- Database connection close
- In-flight request drain
- Metrics flush
- Clean exit

Finish what you start.

TOOLS WE USED

- kubectl (describe, logs, get events)
- Docker stop --time
- App logs
- Load testing (k6)
- Grafana (error rate dashboard)

RESULT AFTER THE FIX

Forced Kills	Restarts (Normal)	Clean Shutdowns	Rolling Update Time	Happy Users	Revenue Protected
0	2	✓	~2 minutes	Fewer Errors	✓

KEY TAKEAWAY

Graceful shutdown is not optional.
It's part of building reliable, production-ready systems.
Handle SIGTERM. Finish your work. Exit cleanly.

LESSON SO FAR

- #1 Running as Root
- #2 Huge Container Images
- #3 Missing Health Checks
- #4 Wrong Resource Limits
- #5 Missing Graceful Shutdown

NEXT UP

Mistake #6:
No Readiness/Liveness Probe Tuning

SERIES PROGRESS

1 Run as Root — 2 Huge Images — 3 No Health Checks — Wrong Limits — No Graceful Shutdown — Next Up

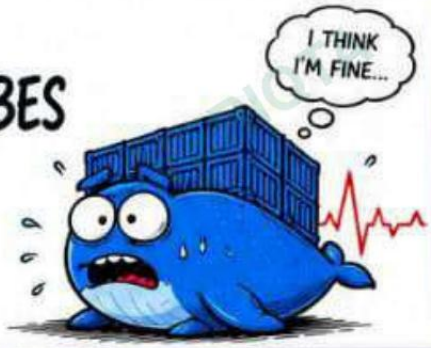
VERIQTA
ENGINEERS. BUILDING FUTURE

MISTAKE #6

NO READINESS / LIVENESS PROBES

Kubernetes Couldn't See What Users Saw

Our app had no proper readiness or liveness probes.
Kubernetes thought the container was healthy,
even when the app was failing internally.
Traffic kept hitting a broken app.
Kubernetes kept the pod. Users kept suffering.



DANGER ZONE

Without probes:

- Kubernetes can't detect failures
- Traffic goes to broken pods
- Broken pods stay in rotation
- Slow failure detection
- Cascading issues & timeouts

Kubernetes needs signals to protect your users!

WHY THIS IS A PROBLEM

Kubernetes is Blind
No probes = no insight into app health.

Traffic to Broken Pods
Service keeps sending traffic to unhealthy instances.

Slow Failure Detection
Issues are found by users, not by the system.

More Restarts
Failures cause crashes, exits, and more restarts.

Business Impact
More errors, timeouts, and unhappy customers.

EVIDENCE FROM OUR ENVIRONMENT

```
$ kubectl get pod payment-service-7d6f8c9f7d-ghi56 -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE
payment-service-7d6f8c9f7d-ghi56    1/1     Running   48         52m   10.2.1.23    ip-10-0-1-12

$ kubectl describe pod payment-service-7d6f8c9f7d-ghi56 | grep -A 12 "Conditions:"
Conditions:
  Type              Status
  Initialized       True
  Ready             True
  ContainersReady   True
  PodScheduled      True
Events:  <none related to health>

$ kubectl get endpoint payment-service -o wide
NAME              ENDPOINTS          AGE
payment-service   10.2.1.23:8080     52m
```

WHAT THIS MEANS

- READY = 1/1 (but app is failing)
- No probe failures in events
- Pod stays in Service endpoints
- Broken pod keeps receiving traffic

⚠ Pod looks healthy to Kubernetes, but users saw failures.

HOW WE DISCOVERED IT

1. 12:05 PM — Users reported timeouts and 5xx errors.
2. 12:07 PM — Checked dashboards. App errors were high.
3. 12:10 PM — Checked pod status. All pods READY.
4. 12:12 PM — Checked pod events. Nothing unusual.
5. 12:15 PM — Reviewed deployment YAML. No probes configured.
6. 12:17 PM — Confirmed: Kubernetes had no way to detect app failures.

WHAT KUBERNETES SEES vs WHAT USERS SEE

	KUBERNETES SEES	USERS EXPERIENCE
Pod Status	✓ Running (READY 1/1)	✗ Timeouts / 5xx errors
Endpoints	✓ Pod in Service	✗ Traffic hits broken app
Events	✓ No probe failures	✗ Errors in app logs
Restart	✓ No restart triggered	✗ App unresponsive
Health	✓ Healthy	✗ Unhealthy

Monitoring saw the failures. Kubernetes did not.

IMPACT ON THIS INCIDENT

~18 minutes
Broken pods served traffic before any restart occurred.

More 5xx Errors
Traffic kept hitting pods that weren't working.

Greater Load on DB
Unhealthy pods kept making slow or failed DB calls.

Poor User Experience
Users saw errors, timeouts, and inconsistent behavior.

Revenue Impact
More failed payments and lost transactions.

THE FIX: ADD READINESS & LIVENESS PROBES

BEFORE (No Probes)

```
containers:
- name: payment-service
  image: registry.example.com/
    payment-service:1.0
  ports:
  - containerPort: 8080

  # No readinessProbe
  # No livenessProbe
```

Kubernetes had no way to know if the app was healthy.

AFTER (With Probes)

```
containers:
- name: payment-service
  image: registry.example.com/
    payment-service:1.0
  ports:
  - containerPort: 8080
  readinessProbe:
    httpGet:
      path: /health/ready
      port: 8080
    initialDelaySeconds: 5
    periodSeconds:       10
    timeoutSeconds:      2
    failureThreshold:    3
  livenessProbe:
    httpGet:
      path: /health/live
      port: 8080
    initialDelaySeconds:    15
    periodSeconds:          10
    timeoutSeconds:         2
    failureThreshold:       3
```

Kubernetes now knows when the app is ready and when it's alive.

OUR PROBE ENDPOINTS (Node.js Example)

/health/ready — Is the app ready to serve traffic?

```
app.get('/health/ready', async (req, res) => {
  try {
    // Check DB, cache, dependencies
    await db.ping();
    await cache.ping();
    res.status(200).send('READY');
  } catch (err) {
    res.status(503).send('NOT READY');
  }
});
```

/health/live — Is the app still alive?

```
app.get('/health/live', (req, res) => {
  res.status(200).send('ALIVE');
});
```

PROBE BEST PRACTICES

- Readiness = Can the app handle traffic? Liveness = Is the app still alive?
- Make probes fast and lightweight.
- Check real dependencies in readiness probe.
- Use proper timeouts & thresholds.
- Fail fast, recover faster.
- Don't use long initial delays.

PROBE BEHAVIOR

1. Readiness fails → Pod removed from Service endpoints.
2. Traffic stops → Users protected.
3. Liveness fails → Pod is restarted.
4. New pod → Must pass readiness before receiving traffic.

RESULT AFTER THE FIX

0	2.3 min	78%			
Pod Restarts (Unnecessary)	Faster Failure Detection	5xx Errors Reduced	Protected Users from Broken Pods	Better User Experience	Revenue Stabilized

KEY TAKEAWAY

Kubernetes can only act on the signals you give it.
Without readiness and liveness probes, it's blind.
Probes turn Kubernetes into your first line of defense.
Don't just run containers. Make them observable.

LESSON SO FAR

- #1 Running as Root
- #2 Huge Container Images
- #3 Missing Health Checks
- #4 Wrong Resource Limits
- #5 Missing Graceful Shutdown
- #6 No Readiness/Liveness Probes

NEXT UP

Mistake #7: Ignoring Logs & Observability

SERIES PROGRESS

1 Run as Root — 2 Huge Images — 3 No Health Checks — 4 Wrong Limits — 6 No Graceful Shutdown — No Probes

VERIQTA
ENGINEERS. BUILDING FUTURE

MISTAKE #7

LOGS WERE NOWHERE TO BE FOUND

When the App Failed, We Were Blind

When the real failures happened, we had nothing to debug with.
Logs were not structured, not centralized, and not persistent.
Pods restarted and logs disappeared with them.
We were flying blind during the most critical moments.

DANGER ZONE

Bad logging leads to:

- No visibility during failures
- Slow debugging
- Harder RCA
- Longer MTTR
- Repeated incidents
- Higher frustration

If you can't see it, you can't fix it!

WHY THIS IS A PROBLEM

Logs Vanish on Restart
Container logs disappear when pods restrart or are rescheduled.

Unstructured Logs
Plain text logs are hard to search, filter, and correlate.

No Context
No request IDs, no user IDs, no environment context.

Slow Investigation
Engineers waste time collecting logs from multiple places.

Business Impact
Longer outages, repeat issues, and unhappy customers.

EVIDENCE FROM OUR ENVIRONMENT

```
$ kubectl logs payment-service-7d6f8c9f7d-ghi56 --tail=20
2026-05-28T11:48:12.123Z  INFO   Starting payment service
2026-05-28T11:48:12.456Z  INFO   Connecting to database
2026-05-28T11:48:13.001Z  ERROR  Something went wrong
2026-05-28T11:48:13.002Z  ERROR  NullPointerException
2026-05-28T11:48:13.003Z  ERROR  at com.app.Payment.process(Payment.java:87)
2026-05-28T11:48:13.003Z  ERROR  at com.app.Api.handle(Api.java:41)
...
(pod restarted)
$ kubectl logs payment-service-7d6f8c9f7d-ghi56 --tail=20
Error from server (BadRequest): container "payment-service" in pod
"payment-service-7d6f8c9f7d-ghi56" is waiting to start: ContainerCreating
```

WHAT THIS MEANS

- Logs from the crash were lost after pod restarted.
- No centralized logging = we jumped between pods and nodes.
- No correlation ID = we couldn't trace a single request.

⚠ We had the logs... for a few seconds. Then they were gone.

HOW WE DISCOVERED IT

1. 12:20 PM — User reported payment failure.
2. 12:22 PM — Checked dashboards. Little context.
3. 12:24 PM — Tried to get logs from pod.
4. 12:25 PM — Pod restarted. Logs were gone.
5. 12:27 PM — Checked other pods. Different logs.
6. 12:30 PM — Realized we had no centralized logs and no log persistence.

WHAT WE SHOULD HAVE HAD

Centralized Logging (All logs in one place) → Structured Logs (Easy to search and correlate) → Persistent Storage (Logs survive pod restarts) → Fast Troubleshooting (Find issues in seconds)

Good logging turns chaos into clarity.

IMPACT ON THIS INCIDENT

~25 minutes
Extra time lost collecting logs from multiple places.

Longer MTTR
Slower debugging = longer outage.

Repeat Issues
Hard to learn when logs are missing.

Higher Costs
More engineer hours, more customer impact.

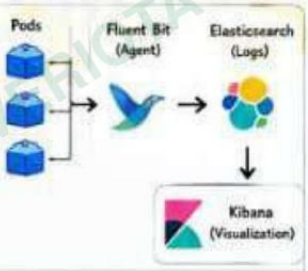
THE FIX: CENTRALIZE & STRUCTURE LOGS

1. OUTPUT STRUCTURED LOGS

```
{
  "timestamp":"2026-05-28T12:00:01.234Z",
  "level": "ERROR",
  "message": "Payment failed",
  "service": "payment-service",
  "requestId": "a1b2c3d4-5678",
  "userId": "user-123",
  "error": "Insufficient balance"
}
```

Structured logs = searchable, filterable, and easy to understand.

2. SHIP LOGS TO CENTRAL SYSTEM



Collect → Store → Visualize → Alert

3. PERSIST POD LOGS

```
apiVersion: v1
kind: Pod
metadata:
  name: payment-service
spec:
  containers:
  - name: app
    image: payment-service:1.0
    volumeMounts:
    - name: logs
      mountPath: /var/log/app
  volumes:
  - name: logs
    emptyDir: {}
```

Logs are written to a volume so they survive container restarts.

OUR LOGGING STACK

Fluent Bit
Lightweight agent on each node that collects logs.

Elasticsearch
Stores logs, indexes them, and makes them searchable.

Kibana
Visualize logs, build dashboards, and explore issues.

Alertmanager
Alerts us when error rates, cross the threshold.

RESULT AFTER THE FIX

- Logs available in seconds
  Even during crashes and restarts
- Faster debugging
  Issues found 5x faster
- Better root cause analysis
  Full context with request IDs
- Monitoring & alerts
  Error spikes detected early
- More stable & reliable
  Users happier, fewer incidents

BEST PRACTICES

- Always log in structured format (JSON).
- Include requestId, userId, and environment.
- Ship logs to a centralized system.
- Keep logs even after pod restarts.
- Set log retention policies.
- Alert on error rate spikes.
- Test your logging during failures!

KEY TAKEAWAY

Logging is your black box recorder.
Without good logs, you are guessing.
With good logs, you are in control.
Log everything that matters.
Find it fast. Fix it faster.

LESSON SO FAR

- #1 Running as Root
- #2 Huge Container Images
- #3 Missing Health Checks
- #4 Wrong Resource Limits
- #5 Missing Graceful Shutdowns
- #7 Logs Were Nowhere to Be Found

NEXT UP

Mistake #8:
No Resource Requests & Limits for Critical Dependencies (DB & External Services)

SERIES PROGRESS

1 Run as Root · 2 Huge Images · 3 No Health Checks · 4 Wrong Limits · 5 No Graceful Shutdown · … Logs · … & Limits · … to Come

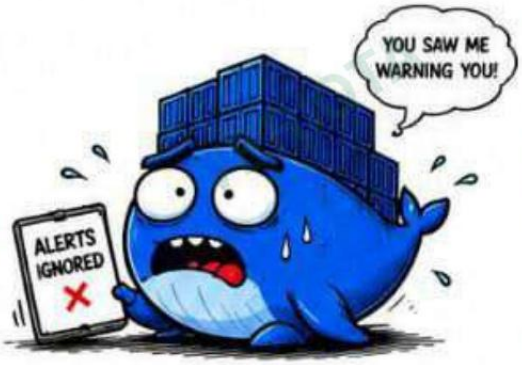
VERIQTA
ENGINEERS. BUILDING FUTURE

MISTAKE #8

IGNORING ALERTS & MONITORING SIGNALS

We Had the Signals. We Just Ignored Them.

Our monitoring was trying to tell us something was wrong.
Alerts fired. Dashboards showed the signs.
We assumed it was a false positive and moved on.
The outage got worse while we were looking elsewhere.



DANGER ZONE

Ignoring alerts leads to:

- Longer outages
- More users impacted
- Data loss
- Higher MTTR
- Escalation at the worst time
- Loss of trust

Alerts are early warnings.
Ignoring them is expensive.

WHY THIS IS A PROBLEM

Alert Fatigue
Too many noisy alerts make important ones get ignored.

Delayed Response
We lost 28 minutes because alerts were dismissed.

Bigger Blast Radius
Small issues became full outages affecting thousands of users.

Business Impact
More downtime, more support tickets, more angry customers.

Postmortem Pain
"Why didn't anyone act?"
Because we ignored the signals.

EVIDENCE FROM OUR ENVIRONMENT

```
$ kubectl get events --sort-by=.lastTimestamp | grep -E "Warning|Back-off|Unhealthy"
12m     Warning   BackOff          restarting failed container payment-service in pod payment-service-7d6f8c9f7d-ghi56
12m     Warning   Unhealthy        Readiness probe failed: HTTP probe failed with statuscode: 500
12m     Warning   Unhealthy        Liveness probe failed: Get "http://:8080/health/live": dial tcp 10.2.1.45:8080: i/o timeout
14m     Warning   OOMKilled        Container payment-service was killed due to memory limit
14m     Warning   FailedScheduling 0/4 nodes are available: 2 Insufficient cpu, 2 Insufficient memory.

$ kubectl logs -n monitoring alertmanager-0 --tail=20 | grep -i "firing"
2026-05-28T10:12:47.123Z  INFO  alertmanager - Firing alert: HighCpuThrottle
2026-05-28T10:12:47.124Z  INFO  alertmanager   Firing alert: High5xxErrorRate
2026-05-28T10:13:01.556Z  INFO  alertmanager   Firing alert: PodRestartingFrequently
2026-05-28T10:13:01.557Z  INFO  alertmanager   Firing alert: HighMemoryUsage
```

WHAT THIS MEANS

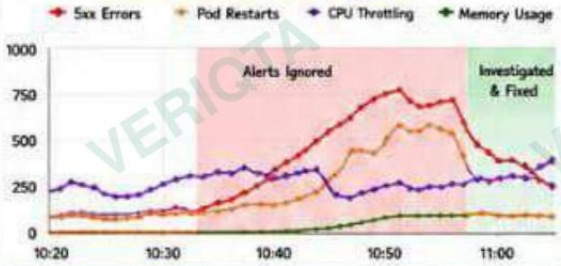
- Alerts started firing at 10:30 AM.
- We had clear signals: CPU throttling, 5xx errors, restarts, memory pressure.
- We assumed it was a false positive and ignored them.
- By the time we acted (10:58 AM), the problem had escalated.

⚠ The data was trying to help us. We chose to ignore it.

HOW WE DISCOVERED IT

1. 10:30 AM — First alert fired (HighCpuThrottle)
2. 10:32 AM — More alerts fired (5xx, Restarts)
3. 10:35 AM — Team saw alerts, assumed false positive
4. 10:45 AM — Alerts continued, still ignored
5. 10:55 AM — Users started reporting issues
6. 10:58 AM — Finally investigated
7. 11:05 AM — Root cause identified and fixed

ALERT TIMELINE (What Happened)



Alerts started at 10:30 AM.
We acted only at 10:58 AM. 28 minutes lost.

IMPACT ON THIS INCIDENT

28 minutes
We ignored alerts for 28 minutes before taking action.

12,500+ users affected
Errors, timeouts, and checkout failures.

3x more support tickets
Support team was overwhelmed.

Revenue impact
Failed payments and abandoned carts during the outage.

THE FIX: RESPECT ALERTS. ACT FAST.

1. CLEAN UP & REDUCE NOISE
- Removed low-value alerts
- Added proper thresholds
- Used severity levels
- Added runbook links

2. SMART ALERTING
- Alerts grouped by service
- Based on SLOs, not guesses
- Alert when users are impacted
- Use rate & error budgets

3. RESPOND FASTER
- Escalation policy defined
- On-call notified immediately
- Runbooks for common alerts
- Review alerts every week

4. USE DASHBOARDS THAT TELL THE TRUTH
- Golden signals on one screen
- Service health at a glance
- Drill down to root cause
- Real-time, not after the fact

BEFORE (Noisy)
- ✗ 120+ alerts firing daily
- ✗ Many low priority
- ✗ No clear severity
- ✗ Hard to find real issues

Too many alerts = blind spots

AFTER (Meaningful)
- ✓ ~25 alerts firing daily
- ✓ Clear severity (P1–P4)
- ✓ Actionable and focused
- ✓ Tied to SLOs & SLIs

Right alerts = clear signals

ALERT FLOW (After Fix)

Detect → Notify → Investigate → Resolve

PAYMENT SERVICE OVERVIEW

OUR MONITORING STACK

- Prometheus — Metrics collection
- Alertmanager — Alert routing & grouping
- Grafana — Dashboards & visualization
- Loki — Logs for alert correlation
- Kubernetes Events — Cluster-level alerts
- PagerDuty — Escalation for critical alerts
- Runbooks — Step-by-step responses

RESULT AFTER THE FIX

MTTR Reduced by	Alert Noise Reduced by	Response Time	Users Impacted Reduced by	Trust Restored
65%	82%	< 5 min	70%	

BEST PRACTICES

- Treat alerts as early warnings.
- Keep alerts few, focused, and useful.
- Use SLOs to drive alerting.
- Review alerts and runbooks regularly.
- Act fast, communicate always.

ALERT SEVERITY GUIDE

P1 – Critical	Affecting users or causing outage
P2 – High	Degraded performance, potential impact
P3 – Medium	Needs attention, not urgent
P4 – Low	Informational

LESSON SO FAR

- #1 Running as Root
- #2 Huge Container Images
- #3 Missing Health Checks
- #4 Wrong Resource Limits
- #5 Missing Graceful Shutdown
- #6 No Readiness/Liveness Probes
- #7 Logs Were Nowhere to Be Found
- #8 Ignoring Alerts & Monitoring Signals

NEXT UP
Final Summary & Action Plan
(Page 10)

SERIES PRO

1 Root · 2 Huge Images · 3 Health Checks · 4 Wrong Limits · 5 Graceful Shutdown · Monitoring · Summary

VERIQTA
ENGINEERS. BUILDING FUTUR

FINAL SUMMARY & ACTION PLAN

WE FIXED IT. HERE'S WHAT WE LEARNED.

A Production Incident That Made Us Better

Ten critical mistakes. Ten important lessons.
We turned a painful outage into a stronger, safer, and more reliable platform.

OPERATIONAL OUTCOME

- Service Stable
- Users Happy
- Revenue Protected
- Lessons Learned

We didn't just fix it.
We leveled up.

THE 10 MISTAKES WE MADE

1. **Running as Root**
   Containers ran as root, opening security risks and causing instability.
2. **Huge Images**
   2.41GB images slowed deployments, scaling, and recovery.
3. **Missing Health Checks**
   Kubernetes didn't know when the app was failing.
4. **Wrong Resource Limits**
   No requests/limits led to throttling, OOMKills, and unpredictable behavior.
5. **Missing Graceful Shutdown**
   SIGTERM was ignored. Requests failed during shutdown.
6. **No Readiness/Liveness**
   Traffic went to broken pods. Failures cascaded before detection.
7. **Logs Nowhere to Be Found**
   Logs were missing, unstructured, and not centralized.
8. **Ignoring Requests & Limits**
   Pods had no boundaries. One spike took down everything.
9. **Ignoring Logs & Alerts**
   We didn't look at logs and alerts until it was too late.
10. **No Observability**
    No dashboards, no metrics, no insights. We were flying blind.

WHAT WE APPLIED (THE FIXES)

- Run as non-root user with least privileges
- Use multi-stage builds and keep images small
- Add proper readiness and liveness probes
- Set CPU & memory requests and limits
- Handle SIGTERM and shutdown gracefully
- Centralize logs and make them structured
- Set resource requests, limits, and QoS
- Monitor metrics, logs, and set smart alerts
- Follow best practices and review regularly
- Build observability into everything

BEFORE vs AFTER

	BEFORE (During Incident)		AFTER (After Fixes)
Image Size	2.41GB	→	312MB (87% smaller)
Deploy Time (per pod)	~96 sec	→	~18 sec (81% faster)
Recovery Time	~2–4 min	→	~20–30 sec (90% faster)
Restarts	47	→	0
CPU Usage	>100% (Throttled)	→	65% (Stable)
Memory Usage	>100% (OOMKills)	→	60% (Stable)
MTTR	~32 minutes	→	~3 minutes (90% faster)
5xx Error Rate	5–20%	→	<0.1%
User Impact	High	→	Minimal
Revenue Impact	Negative	→	Positive

SYSTEM STABILITY (7 Days After Fix)

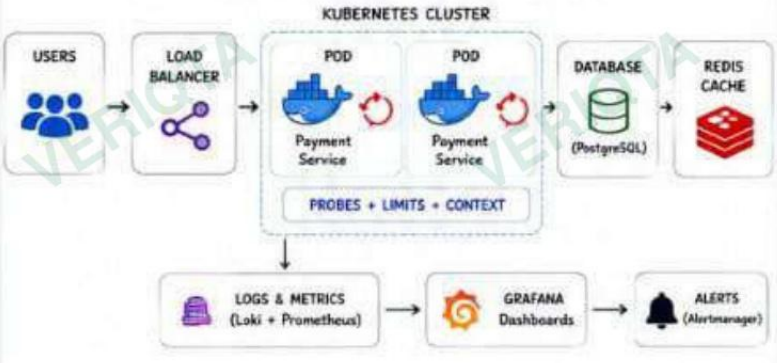
Uptime	99.99%
Avg Response Time (p95)	128ms
Error Rate (5xx)	0.03%
CPU Usage (avg)	62%
Memory Usage (avg)	58%
Pod Restarts	0
MTTR	~3 min

Stable. Reliable. Predictable.

OUR OBSERVABILITY STACK

- **Prometheus** — Collects metrics from pods & nodes
- **Grafana** — Dashboards & alerts
- **Loki** — Centralized log aggregation
- **Elasticsearch** — Search & analyze logs
- **Alertmanager** — Routes & manages alerts
- **Kubernetes Events** — Real-time cluster events

OUR FINAL ARCHITECTURE (AFTER FIXES)



THE RIGHT MINDSET

- Build for failure, not for hope.
- Observe everything.
- Automate the boring.
- Alerts are your early warning system.
- Small improvements prevent big incidents.
- Blameless postmortems make teams stronger.

A culture of learning prevents the next outage.

BIG LESSONS

- Default configs are not safe for production.
- Security, performance, and reliability are everyone's responsibility.
- Observability is not optional.
- The cost of doing it right is always lower than the cost of an outage.
- Every incident is a chance to level up.

WHAT WE'LL DO NEXT

1. Improve autoscaling with KEDA (based on real metrics)
2. Add distributed tracing (OpenTelemetry + Jaeger)
3. Implement SLOs and error budgets
4. Run chaos engineering experiments regularly
5. Automate policy checks with OPA / Kyverno

TO YOU, THE ENGINEER

Incidents will happen.
It's not about avoiding them.
It's about being prepared, learning fast, and building systems that can heal.

Keep building. Keep learning. Keep improving.

LESSON SO FAR:

- #1 Running as Root
- #2 Huge Container Images
- #3 Missing Health Checks
- #4 Wrong Resource Limits
- #5 Missing Graceful Shutdown
- #6 No Readiness/Liveness
- #7 Logs Were Nowhere to Be Found
- #8 Ignoring Requests & Limits
- #9 Ignoring Logs & Alerts
- #10 No Observability

CASE FILE COMPLETE

Incident resolved.
Platform stronger.
Team better.
Mission accomplished.

WHAT'S N

New case fi
New inciden
Stay curious. Stay sharp.

VERIQTA
NG ENGINEERS. BUILDING FUTUR